

A Project Activity Report

Intelligent Transaction Monitoring System for Fraud Detection

Batch : 2Q35

Name of student

Roll number

Bhavya Jain

1024170250.

Naman Bansal

1024170407.

Submitted to

Dr. Diksha Arora.

UCS310 – Database Management Systems
(BTECH 2nd year)



THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY, (A DEEMED TO BE
UNIVERSITY), PATIALA, PUNJAB INDIA

Jan - May 2026

INDEX

S.NO.	TITLE	PAGE NO.
1.	INTRODUCTION	3
2.	PROBLEM STATEMENT	4
3.	ER DIAGRAM	5
4.	ER TO TABLE	6
5.	RELATIONSHIPS	6
6.	NORMALIZATION	7-8
7	TABLE	9-10
8	DASHBOARD	10-11
8.	PL/SQL CODE	12-13
13.	PL/SQL COMPONENTS	14-17
	CONCLUSION	18

INTRODUCTION

The rapid advancement of digital banking and financial technologies has significantly increased the volume of online transactions worldwide. While this has enhanced convenience and accessibility for users, it has also led to a rise in fraudulent activities such as unauthorized transactions, identity theft, and financial scams. Traditional fraud detection systems often rely on manual monitoring or outdated rule-based mechanisms, which are inefficient, time-consuming, and incapable of detecting fraud in real time.

To address these challenges, this project presents FRAUD GUARD: An Intelligent Transaction Monitoring System for Fraud Detection using SQL and Triggers. The system is designed using a Database Management System (DBMS) approach to efficiently store, process, and analyse transaction data. It leverages structured query language (SQL), stored procedures, and triggers to automatically detect suspicious transactions based on predefined rules and behavioural patterns. The system incorporates multiple fraud detection techniques, including high-value transaction monitoring, rapid transaction detection, unusual transaction timing, and location-based anomaly detection. Additionally, it maintains a user transaction pattern profile to identify deviations from normal behaviour, thereby improving detection accuracy.

By implementing automated detection mechanisms at the database level, FRAUD GUARD ensures faster response times, improved data integrity, and enhanced security. This system demonstrates how DBMS concepts can be effectively applied to solve real-world financial security challenges.

PROBLEM STATEMENT

The **Fraud Guard System (FGS)** is designed to monitor and detect fraudulent activities in online and ATM-based financial transactions. With the rapid growth of digital banking, the volume of transactions has increased significantly, making it difficult to manually track suspicious activities. Traditional systems often rely on delayed processing and lack real-time monitoring, leading to inefficiencies, increased fraud risk, and delayed response to suspicious transactions.

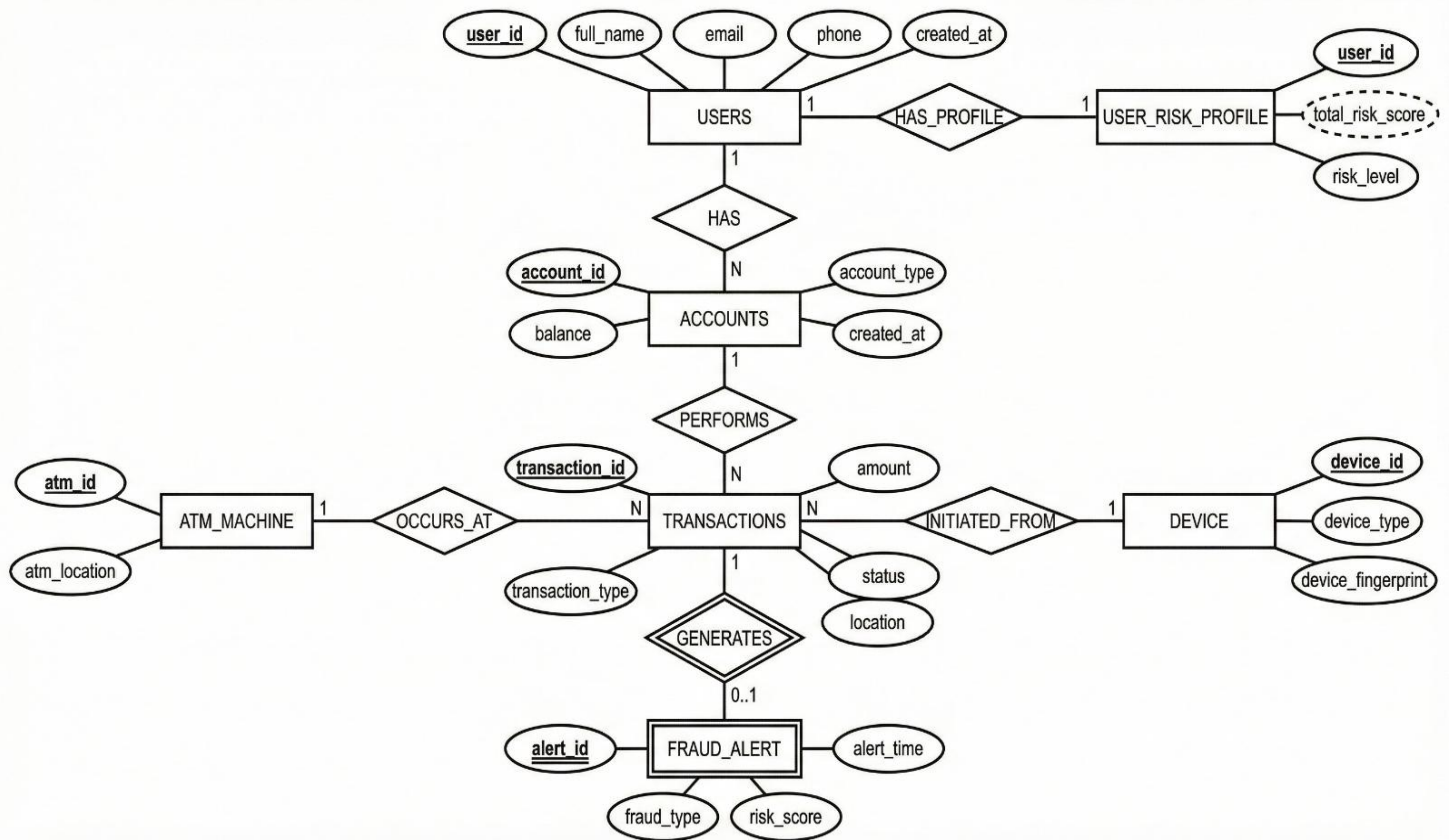
Manual and file-based transaction monitoring systems are not scalable and fail to provide accurate and timely fraud detection. They are prone to human errors, lack consistency, and cannot effectively analyze large volumes of transaction data. Moreover, the absence of automated risk assessment and behaviour tracking makes it difficult to identify unusual transaction patterns.

This project aims to **develop a secure, scalable, and efficient database-driven system** that can automatically monitor transactions, detect fraudulent activities in real time, and maintain user risk profiles. The system uses SQL, triggers, and stored procedures to ensure data integrity, automate fraud detection, and enable effective transaction analysis.

Objectives:

- ❑ **Centralized Transaction Management:** Store and manage user, account, and transaction data in a structured database system.
- ❑ **Real-Time Fraud Detection:** Identify suspicious activities such as rapid transactions, high-value transactions, and unusual transaction timings using triggers and procedures.
- ❑ **Risk Assessment System:** Maintain cumulative risk scores for users based on detected fraud patterns.
- ❑ **Behavior Pattern Analysis:** Track user transaction patterns (average amount, location) to detect anomalies.
- ❑ **Data Integrity & Automation:** Use triggers and constraints to ensure accurate and consistent data processing.
- ❑ **Efficient Querying & Reporting:** Enable analysis of transaction data using SQL queries for better decision-making and monitoring.

ER DIAGRAM



ER TO TABLE

- Users (user_id, full_name, email, phone, created_at)
- Accounts (account_id, user_id (fk), account_type, balance, status)
- Transactions (transaction_id, account_id (fk), transaction_type, amount, location, transaction_time, status)
- Fraud_Alerts (alert_id, transaction_id (fk), fraud_type, risk_score, alert_time)
- User_Risk_Profile (user_id (pk, fk), total_risk_score, risk_level)
- User_Transaction_Pattern (user_id (pk, fk), avg_amount, max_amount, usual_location, last_updated)

RELATIONSHIPS

One User can have multiple Accounts (One-to-Many)

- One Account can perform multiple Transactions (One-to-Many)
- One Transaction can generate multiple Fraud Alerts (One-to-Many)
- Each User has one Risk Profile (One-to-One)
- Each User has one Transaction Pattern record (One-to-One)

NORMALIZATION

Introduction to Normalization

Normalization is a database design technique that aims to reduce data redundancy and ensure data integrity. It involves organizing the fields and tables of a relational database to minimize dependency and repetition. The process involves decomposing large tables into smaller, well-structured ones without losing any data.

- The main objectives of normalization are:
 - Eliminate redundant data
 - Ensure data dependencies make sense (store only related data in a table)
 - Simplify data modification and ensure consistency

The normalization process involves multiple normal forms, primarily:

- **1NF (First Normal Form)** – Eliminates repeating groups
- **2NF (Second Normal Form)** – Removes partial dependency
- **3NF (Third Normal Form)** – Removes transitive dependency

Un-normalized table

Before normalization, transaction data may contain repeated user and account details along with transaction records, leading to redundancy and inconsistency.

Application of Normalization in Our System

The FRAUD GUARD system is designed using normalization principles. All tables are structured

up to **Third Normal Form (3NF)** to ensure efficient storage, minimal redundancy, and reliable data processing. Heres how-

First Normal Form (1NF)

A relation is in 1NF if:

- It contains only atomic (indivisible) values.

- **Application:**

- In the **Users** table, attributes like name, email, and phone store atomic values
- In the **Transactions** table, each field (amount, location, time) contains a single value
- No repeating groups exist in any table

Second Normal Form (2NF)

A relation is in 2NF if:

- It is in 1NF
- It has no partial dependencies

Application:

- Each table has a single primary key (e.g., user_id, account_id), so all non-key attributes depend fully on the primary key
- For example, in the **Accounts** table, balance and status depend entirely on account_id

Third Normal Form (3NF)

A relation is in 3NF if:

- It is in 2NF
- There are no transitive dependencies

Application:

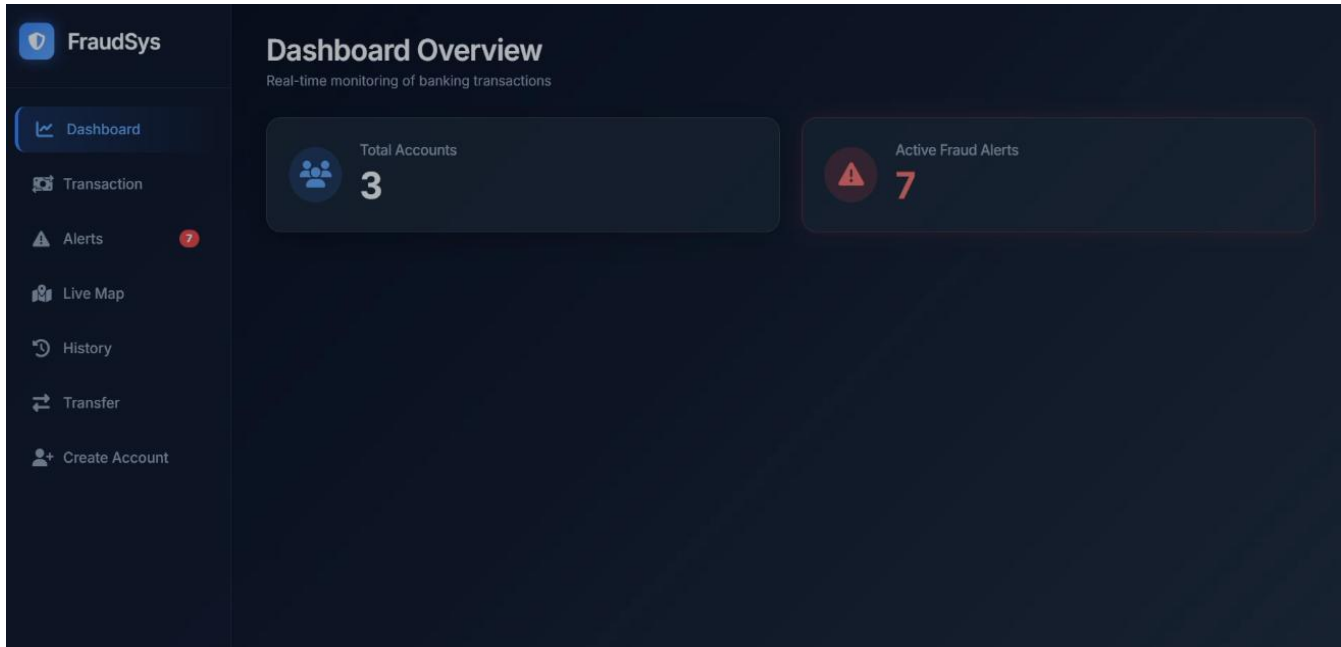
- User details are stored separately in the **Users** table instead of being repeated in Accounts or Transactions
- Fraud-related data is stored separately in the **Fraud_Alerts** table
- Behavioral data is maintained independently in the **User_Transaction_Pattern** table

Advantages of Normalization in This System

- Data Integrity
- Reduced Redundancy
- Improved Query Performance
- Easier Maintenance

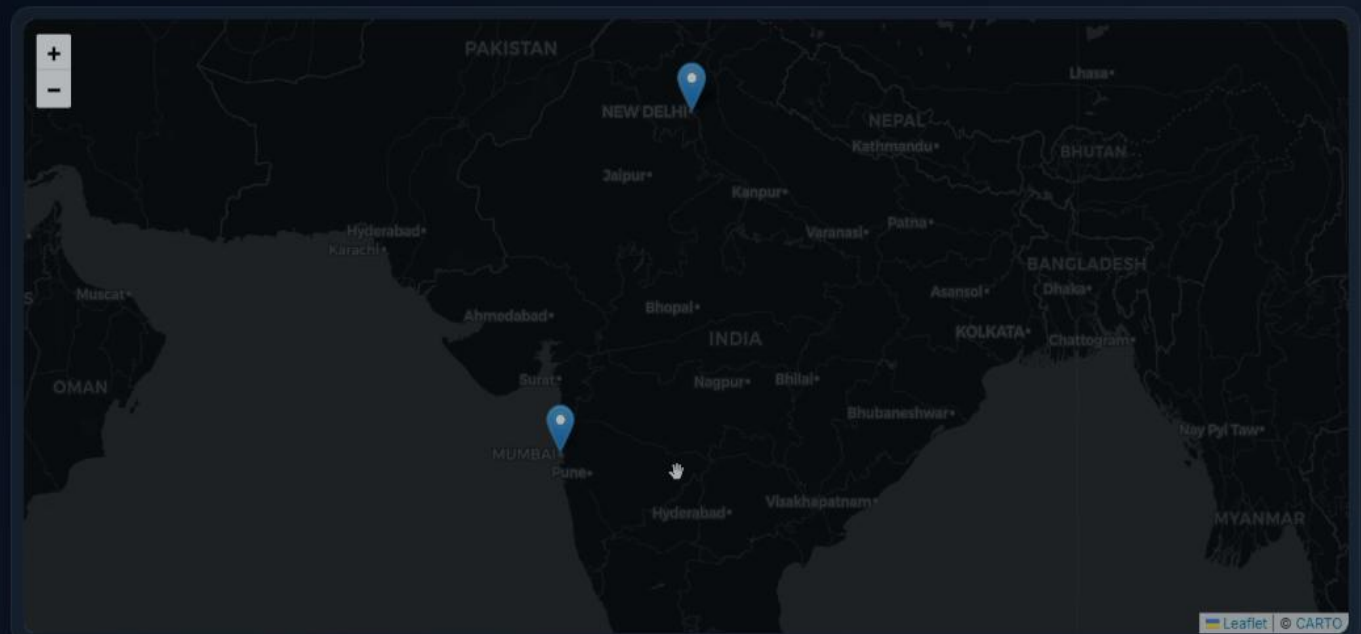
TABLE NAME	DESCRIPTION
Users	Stores personal details of users such as name, email, and phone number. Each record is uniquely identified by user_id .
Accounts	Contains bank account details linked to users. Maintains referential integrity using user_id as a foreign key.
Transactions	Stores all transaction records including amount, type, time, and location. Each transaction is uniquely identified by transaction_id .
Fraud_Alerts	Records all detected fraudulent activities along with fraud type and risk score. Linked to transactions using transaction_id . Records all detected fraudulent activities along with fraud type and risk score. Linked to transactions using transaction_id .
User_Risk_Profile	Maintains cumulative risk score and risk level for each user. Each user has exactly one risk profile identified by user_id .
User_Transaction_Pattern	Stores behavioral patterns of users such as average transaction amount, maximum amount, and usual location. Used for pattern-based fraud detection.
(Triggers & Procedures Logic)	Implements fraud detection logic, validation, and automation using triggers and stored procedures to ensure real-time monitoring and data integrity.

FRAUD GUARD INTERFACE



Live Transaction Map

Geographical distribution of recent alerts



FraudSys

Dashboard

Transaction

Alerts0

Live Map

History

Make Transaction

Simulate a new banking transaction

Account ID

e.g. ACC1001

Amount (\$)

\$ e.g. 500.00

Location

e.g. New York, USA

Transaction Type

Online Transfer

Submit Transaction

FraudSys

Dashboard

Transaction

Alerts7

Live Map

History

Transfer

Create Account

Fraud Alerts

Recent suspicious activities detected

High Amount

Risk: 80%

Remote

Account ID: 3

Sudden Location Change

Risk: 50%

Remote

Account ID: 3

Pattern Amount Deviation

Risk: 50%

Remote

Account ID: 3

Insufficient Balance Attempt

Risk: 85%

Remote

Account ID: 3

Sudden Location Change

Risk: 50%

USA

Account ID: 3

Pattern Location Deviation

Risk: 40%

USA

Account ID: 3

High Amount

Risk: 60%

Mumbai

Account ID: 1

11

PL/SQL CODES

CREATING TABLES:

1. USERS

```
CREATE TABLE users (  
    user_id INT PRIMARY KEY AUTO_INCREMENT,  
    full_name VARCHAR(100),  
    email VARCHAR(100) UNIQUE,  
    phone VARCHAR(15),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

2. ACCOUNTS

```
CREATE TABLE accounts (  
    account_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT,  
    account_type VARCHAR(20),  
    balance DECIMAL(12,2) DEFAULT 0,  
    status VARCHAR(20) DEFAULT 'ACTIVE',  
    FOREIGN KEY (user_id) REFERENCES users(user_id)  
);
```

3. TRANSACTIONS:

```
CREATE TABLE transactions (  
    transaction_id INT PRIMARY KEY AUTO_INCREMENT,  
    account_id INT,  
    transaction_type VARCHAR(20),  
    amount DECIMAL(10,2),  
    location VARCHAR(100),  
    transaction_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    status VARCHAR(20),  
    FOREIGN KEY (account_id) REFERENCES accounts(account_id)  
);
```

4 FRAUD ALERTS

```
CREATE TABLE fraud_alerts (  
  alert_id INT PRIMARY KEY AUTO_INCREMENT,  
  transaction_id INT,  
  fraud_type VARCHAR(100),  
  risk_score INT,  
  alert_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (transaction_id) REFERENCES transactions(transaction_id)  
);
```

5- USER RISK PROFILE:

```
CREATE TABLE user_risk_profile (  
  user_id INT PRIMARY KEY,  
  total_risk_score INT DEFAULT 0,  
  risk_level VARCHAR(20),  
  FOREIGN KEY (user_id) REFERENCES users(user_id)  
);
```

6- PATTERN MEMORY:

```
CREATE TABLE user_transaction_pattern (  
  user_id INT PRIMARY KEY,  
  avg_amount DECIMAL(10,2),  
  max_amount DECIMAL(10,2),  
  usual_location VARCHAR(100),  
  last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(user_id)  
);
```

PL/SQL Components (Stored Programs + Triggers)

PL/SQL COMPONENTS

PL/SQL is used to implement the business logic of the FRAUD GUARD system. It enables automated fraud detection, data validation, and behavior tracking within the database.

1. Function: Risk Calculator

A user-defined function is created to calculate the risk score of a transaction based on its amount. Higher transaction amounts are assigned higher risk values, helping in identifying potentially fraudulent activities.

```
DELIMITER $$

DROP FUNCTION IF EXISTS calculate_risk $$

CREATE FUNCTION calculate_risk(amount DECIMAL(10,2))
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE risk INT;

    IF amount > 50000 THEN
        SET risk = 80;
    ELSEIF amount > 20000 THEN
        SET risk = 60;
    ELSEIF amount > 10000 THEN
        SET risk = 40;
    ELSE
        SET risk = 10;
    END IF;

    RETURN risk;
END$$

DELIMITER ;
```

2. Procedure: Update User Transaction Pattern

This procedure updates the behavioral pattern of users by calculating their average transaction amount, maximum transaction amount, and most frequent transaction location. This information is stored in the user transaction pattern table and is used for detecting anomalies.

```
-----  
-- PROCEDURE: UPDATE USER PATTERN  
-----  
  
DELIMITER $$  
  
DROP PROCEDURE IF EXISTS update_user_pattern $$  
  
CREATE PROCEDURE update_user_pattern(IN p_user_id INT)  
BEGIN  
    DECLARE avg_amt DECIMAL(10,2);  
    DECLARE max_amt DECIMAL(10,2);  
    DECLARE loc VARCHAR(100);  
  
    SELECT AVG(t.amount), MAX(t.amount), t.location  
    INTO avg_amt, max_amt, loc  
    FROM transactions t  
    JOIN accounts a ON t.account_id = a.account_id  
    WHERE a.user_id = p_user_id  
    GROUP BY t.location  
    ORDER BY COUNT(*) DESC  
    LIMIT 1;  
  
    INSERT INTO user_transaction_pattern (user_id, avg_amount, max_amount, usual_location)  
    VALUES (p_user_id, avg_amt, max_amt, loc)  
    ON DUPLICATE KEY UPDATE  
        avg_amount = avg_amt,  
        max_amount = max_amt,  
        usual_location = loc;  
END$$  
  
DELIMITER ;
```

3. Procedure: Fraud Detection

The main fraud detection procedure analyzes each transaction and identifies suspicious activities based on predefined rules such as rapid transactions, high-value transactions, unusual timing, sudden location changes, and deviation from user behavior patterns. When fraud is detected, alerts are generated and stored in the fraud alerts table.

```
-----  
-- MAIN PROCEDURE: FRAUD DETECTION  
-----  
DELIMITER $$  
  
DROP PROCEDURE IF EXISTS detect_fraud $$  
  
CREATE PROCEDURE detect_fraud(IN p_transaction_id INT)  
BEGIN  
    DECLARE txn_account_id INT;  
    DECLARE txn_amount DECIMAL(10,2);  
    DECLARE txn_location VARCHAR(100);  
    DECLARE txn_count INT;  
    DECLARE txn_hour INT;  
  
    DECLARE avg_amt DECIMAL(10,2);  
    DECLARE usual_loc VARCHAR(100);  
    DECLARE last_loc VARCHAR(100);  
  
    -- Get transaction details  
    SELECT account_id, amount, location  
    INTO txn_account_id, txn_amount, txn_location  
    FROM transactions  
    WHERE transaction_id = p_transaction_id;  
  
    -- Rapid Transactions  
    SELECT COUNT(*) INTO txn_count  
    FROM transactions  
    WHERE account_id = txn_account_id  
    AND transaction_time >= NOW() - INTERVAL 5 MINUTE;  
  
    IF txn_count > 3 THEN  
        INSERT INTO fraud_alerts (transaction_id, fraud_type, risk_score)  
        VALUES (p_transaction_id, 'Rapid Transactions', 40);  
    END IF;  
  
    -- High Amount  
    IF txn_amount > 20000 THEN  
        INSERT INTO fraud_alerts (transaction_id, fraud_type, risk_score)  
        VALUES (p_transaction_id, 'High Amount', calculate_risk(txn_amount));  
    END IF;  
  
    -- Time-based Fraud  
    SET txn_hour = HOUR(NOW());  
  
    IF txn_hour BETWEEN 0 AND 5 THEN  
        INSERT INTO fraud_alerts (transaction_id, fraud_type, risk_score)  
        VALUES (p_transaction_id, 'Odd Time Transaction', 30);  
    END IF;  
END
```


4. Triggers

Triggers are used to automate system operations:

- A **BEFORE INSERT trigger** ensures that invalid transactions (e.g., negative amount) are not allowed.
- An **AFTER INSERT trigger** automatically invokes the fraud detection procedure and updates user patterns whenever a new transaction is recorded.

```
-- TRIGGERS
-----

-- Validation
DELIMITER $$

DROP TRIGGER IF EXISTS before_transaction_insert $$

CREATE TRIGGER before_transaction_insert
BEFORE INSERT ON transactions
FOR EACH ROW
BEGIN
    IF NEW.amount <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Invalid transaction amount';
    END IF;
END$$

DELIMITER ;

-- Main trigger
DELIMITER $$

DROP TRIGGER IF EXISTS after_transaction_insert $$

CREATE TRIGGER after_transaction_insert
AFTER INSERT ON transactions
FOR EACH ROW
BEGIN
    CALL detect_fraud(NEW.transaction_id);

    CALL update_user_pattern(
        (SELECT user_id FROM accounts WHERE account_id = NEW.account_id)
    );
END$$

DELIMITER ;
```

CONCLUSION

In conclusion, the FRAUD GUARD system developed using SQL and PL/SQL demonstrates the effective application of database technology in the domain of financial security and fraud detection.

By providing structured storage, real-time transaction monitoring, and automated fraud detection mechanisms, the system significantly enhances the accuracy and efficiency of identifying suspicious activities. The use of triggers and stored procedures enables instant analysis of transactions, ensuring timely detection of potential frauds without manual intervention.

The system incorporates advanced features such as risk scoring and user behavior pattern analysis, which improve the reliability of fraud detection by identifying anomalies in transaction patterns. The well-designed relational database ensures data integrity, consistency, and efficient querying, making the system scalable and suitable for handling large volumes of financial data.

Additionally, the integration of a user-friendly interface allows users to perform transactions, view alerts, and analyze transaction history with ease. The real-time visualization of fraud alerts further enhances monitoring capabilities.

Overall, this project highlights the importance of database management systems in building secure, automated, and efficient solutions for modern financial systems, contributing to improved fraud prevention and better decision-making processes

.